

Plano: mapas dos microdados em HTML estático (GitHub Pages)

censo-escolar-analysis

August 18, 2026

Contents

1	Objetivo	2
2	O comando	3
3	As regras de geração	3
3.1	R1 — A URL carrega três informações, e o gerador lê as três	3
3.2	R2 — <code>intrarregiao</code> determina a coluna de agrupamento . . .	4
3.3	R3 — A junção é direta: <code>codarea</code> coluna agrupadora	4
3.4	R4 — A variável decide a agregação e o tipo de mapa	4
3.5	R5 — Os anos filtram <code>NU_ANO_CENSO</code> , e a disponibilidade é conferida antes	5
3.6	R6 — O nome do mapa é determinístico	6
3.7	R7 — O produto é um GeoJSON com o dado já dentro	6
3.8	R8 — O <code>make map</code> também registra a consulta	7
3.9	R9 — As regras de robustez	7
3.10	Exemplo completo, do comando à página	8
4	Arquitetura e arquivos	8
4.1	O que entra no git e o que não entra	9
4.2	JavaScript	9
5	Comandos make	9
6	Fatos verificados	10
6.1	Os microdados não têm latitude/longitude	10
6.2	Até onde o drill-down chega	10

6.3	O buraco de 2025	11
6.4	Custo das malhas (medido, gzip que o Pages já serve)	11
6.5	O que o projeto já tem e será reaproveitado	11
7	Decisões registradas	12
8	Fases	12
9	Riscos e limites	13
10	Critérios de verificação	13
11	Histórico das decisões	13

Este documento é o plano dos mapas. Ele foi consolidado em 2026-08-18 depois de três rodadas de conversa; o *Histórico das decisões*, no fim, guarda o caminho percorrido — inclusive as ideias descartadas e o motivo.

A ideia central cabe em uma linha de comando:

```
make map QT_MAT_BAS "https://servicodados.ibge.gov.br/api/v3/malhas/estados/35?formato=
```

Uma variável do INEP + uma URL de malha do IBGE = um mapa publicado. Todo o resto deste documento explica as regras que transformam esses dois argumentos numa página HTML.

1 Objetivo

Publicar mapas dos microdados do Censo Escolar como HTMLs estáticos no GitHub Pages:

- um `index.html` que lista os mapas disponíveis;
- cada mapa é uma página que carrega um único arquivo de dados e desenha, sem servidor por trás;
- os mapas são *assados* por comandos `make` reproduzíveis, e a mescla de vários anos usa a coluna `NU_ANO_CENSO`;
- o visitante não monta consultas: ele escolhe um mapa da lista. Quem monta a consulta é quem roda o `make`.

2 O comando

```
make map <VARIABEL> "<URL da malha>" [anos...] [OPCAO=valor...]
```

Argumento	Obrigatório	Exemplo
variável do INEP	sim	QT_MAT_BAS, IN_INTERNET
URL da malha IBGE	sim	"https://.../estados/35?...&intrarregiao=microrregiao"
ano(s)	não	2023, ou 2019 2024 (padrão: o mais recente que serve)
CATEGORIA	só p/ TP_	CATEGORIA=4 (privada)
TITULO	não	TITULO"Matrículas na básica — SP"=

As aspas na URL são obrigatórias. Isso é do shell, não do make: sem elas, o & da URL manda o comando para segundo plano e nada roda.

Exemplos:

```
make map QT_MAT_BAS "https://.../estados/35?...&intrarregiao=microrregiao" 2023
make map IN_INTERNET "https://.../paises/BR?...&intrarregiao=UF" 2019 2024
make map TP_DEPENDENCIA "https://.../estados/31?...&intrarregiao=municipio" 2023 CATEGORIA=4
```

3 As regras de geração

Esta é a seção central: como os dois argumentos viram um mapa. Cada regra tem a evidência que a sustenta — tudo abaixo foi verificado na API do IBGE e nos microdados em disco, em 2026-08-18.

3.1 R1 — A URL carrega três informações, e o gerador lê as três

<https://servicodados.ibge.gov.br/api/v3/malhas/estados/35?formato=...&qualidade=minima>

	recorte espacial	custo/detalhe
Parte da URL	Vira	
caminho (/estados/35)	filtro das linhas dos microdados	
intrarregiao	coluna de agrupamento da agregação	
qualidade	tamanho do arquivo (só afeta o peso)	
formato	normalizado pelo gerador (veja R9)	

O caminho vira filtro assim:

Caminho	Filtro aplicado
/paises/BR	nenhum (Brasil inteiro)
/estados/35	CO_UF = 35=
/municipios/3550308	CO_MUNICIPIO = 3550308=

3.2 R2 — intrarregiao determina a coluna de agrupamento

Tabela verificada na API (contagem de feições e formato de `codarea` reais):

intrarregiao	codarea	Coluna agrupadora	Feições	Anos em que a coluna e
(ausente, em /estados/35)	35	CO_UF	1	todos
UF	11	CO_UF	27	todos
regiao-intermediaria	1102	CO_REGIAO_GEOG_INTERM	133	2023–2025
mesorregiao	1102	CO_MESORREGIAO	137	2019–2024
regiao-imediata	110005	CO_REGIAO_GEOG_IMED	510	2023–2025
microrregiao	35035	CO_MICRORREGIAO	558	2019–2024
municipio	1100015	CO_MUNICIPIO	5.570	todos

Armadilha registrada: mesorregião e região intermediária usam ambas códigos de 4 dígitos — 1102 é uma de cada, e são divisões diferentes. Não dá para inferir o grão pelo formato do código; tem de se ler o `intrarregiao` da URL. É um bom motivo a mais para a URL ser a consulta.

Limite da API (testado, o erro é explícito): em `/estados/XX` só valem `mesorregiao`, `microrregiao` e `municipio`; `/municipios/XXXXXXX` não aceita `intrarregiao` nenhum. Distrito não é servido por esta API — veja "Até onde o drill-down chega".

3.3 R3 — A junção é direta: `codarea` coluna agrupadora

O `properties.codarea` de cada polígono casa com o valor da coluna agrupadora **sem conversão**. Conferido na URL de exemplo: a malha devolve `codarea = "35035"` e uma escola de São Paulo em 2023 traz `CO_MICRORREGIAO = 35061` — mesmo formato, mesma origem (IBGE).

O gerador compara os dois conjuntos e **relata a cobertura**: quantos polígonos ficaram sem dado e quantos códigos de dado não acharam polígono. Um mapa com buracos silenciosos é pior que um erro.

3.4 R4 — A variável decide a agregação e o tipo de mapa

Quem responde isso é o dicionário de dados que o projeto **já gera** (`censo dicionario` → `data/processed/dicionario_colunas_comuns.csv`), na col-

una **natureza**. Contagem do arquivo atual:

natureza	Nº	Exemplo	Agregação por área	Mapa
contagem (QT_)	168	QT_MAT_BAS	soma	coroplético do total
indicador 0/1 (IN_)	282	IN_INTERNET	média → % das escolas	coroplético de taxa (0–100)
categórica (TP_)	17	TP_DEPENDENCIA	% da categoria pedida	coroplético da categoria
código/nome/data/etc.	34	CO_UF, NO_*	—	recusa, com explicação

São ~467 variáveis que viram mapa sem código novo. Duas ressalvas que o gerador aplica sozinho:

- TP_* exige CATEGORIA=; sem ela, o padrão é pintar a **categoria dominante** de cada área;
- toda página ganha, no rodapé, o **descricao** oficial do INEP para aquela variável — texto que o dicionário já traz.

3.5 R5 — Os anos filtram NU_ANO_CENSO, e a disponibilidade é conferida antes

Anos soltos na linha de comando (2023, ou 2019 2024) viram filtro sobre NU_ANO_CENSO depois do empilhamento feito por `loading.carregar_anos()`. Sem ano, usa-se o mais recente em que a variável e a coluna agrupadora existem.

Antes de ler qualquer CSV, o gerador cruza três disponibilidades e recusa com mensagem clara se alguma falhar:

1. a **variável** existe naquele ano? (o dicionário sabe: **anos_ausentes**);
2. a **coluna agrupadora** existe naquele ano? (tabela da R2 — pedir **microrregiao** em 2025 ou **regiao-imediata** em 2019 é erro);
3. o **tipo é estável** entre os anos pedidos? Se **tipo_estavel = False**, o mapa é gerado com uma ressalva no rodapé.

Com mais de um ano, a página ganha um seletor e o arquivo de dados traz uma série por área — nada de baixar de novo ao trocar o ano.

3.6 R6 — O nome do mapa é determinístico

`<variavel>-<intrarregiao>-<recorte>-<anos>`, em minúsculas. A URL de exemplo com QT_MAT_BAS e 2023 produz:

```
site/mapas/qt_mat_bas-microrregiao-35-2023.html
site/dados/qt_mat_bas-microrregiao-35-2023.geojson
```

Rodar o mesmo comando de novo **sobrescreve** a mesma página, em vez de acumular lixo. É o que permite reassar o site inteiro sem limpar nada antes.

3.7 R7 — O produto é um GeoJSON com o dado já dentro

Aqui está a simplificação que mais economiza trabalho, e ela decorre de uma decisão sua: *as malhas baixadas ficam no .gitignore*. Se a malha crua não entra no git, ela não pode ser servida ao navegador — então o gerador **funde** dado e geometria numa coisa só:

```
{"type": "FeatureCollection",
  "features": [
    {"type": "Feature",
      "properties": {"codarea": "35035", "nome": "Adamantina",
        "valor": {"2023": 148}, "escolas": {"2023": 148}},
      "geometry": {...}}
  ],
  "meta": {"variavel": "QT_MAT_BAS", "descricao": "Número de Matrículas...",
    "natureza": "contagem", "anos": [2023], "fonte_malha": "<url>",
    "gerado_em": "2026-08-18", "cobertura": "63/63 áreas com dado"}}
```

Consequências:

- **uma requisição por página** (o GeoJSON), em vez de duas (malha + dados);
- só entram no git as áreas efetivamente usadas — o Brasil inteiro por município nunca é servido quando o mapa é de uma UF;
- o **meta** vai junto, então a página se documenta sozinha (fonte, data de geração, cobertura da junção) sem arquivo paralelo;
- o JavaScript encolhe para "buscar um arquivo, pintar, mostrar popup".

E, do mesmo comando, sai também o **CSV agregado** em `data/processed/<slug>.csv` — a tabela que gerou o mapa, conferível no Calc ou com `censo amostra --arquivo`. O CSV existe para inspeção e reuso; o GeoJSON, para o navegador. Saem juntos justamente para nunca divergirem: se o número do mapa parecer errado, a tabela que o produziu está ali do lado.

3.8 R8 — O make map também registra a consulta

Cada geração acrescenta (ou atualiza) uma linha em `site/consultas.txt`, com exatamente o que se digitou:

# variável	URL da malha	anos	opções
QT_MAT_BAS	"https://.../estados/35?...&intrarregiao=microrregiao"	2023	
IN_INTERNET	"https://.../países/BR?...&intrarregiao=UF"	2019	2024

O **site inteiro é função desse arquivo**. `make site` relê a lista e reassa tudo; o `index.html` é gerado dela, nunca editado à mão; apagar um mapa é apagar uma linha; e o `git diff` mostra literalmente quais mapas entraram, saíram ou mudaram. Um mapa publicado passa a ser revisável como código.

3.9 R9 — As regras de robustez

1. **Normalizar o formato=**: se a URL vier sem `formato=application/vnd.geo+json`, o gerador acrescenta — sem isso a API devolve outra representação e o erro só apareceria ao desenhar.
2. **Cache por hash da URL** em `data/cache/malhas/<sha1>.json` (no `.gitignore`): reassar o site não depende da API no ar, e não se martela o IBGE a cada build.
3. **Avisar quando a malha for pesada**: acima de ~150 KiB gzipados, sugerir `qualidade=minima`. Medido nas mesmas 63 microrregiões de SP:

qualidade	Bruto	Gzip
maxima	1,77 MiB	416 KiB
minima	91 KiB	21,7 KiB

Dezenove vezes menor, pelos mesmos polígonos. A malha é fundo de mapa: a precisão de vértice não muda a leitura do dado. É aviso, não imposição — num mapa de uma cidade só, **maxima** pode fazer sentido.

4. **Detectar a URL sem aspas:** se chegar uma URL truncada (sem `intrarregiao/=qualidade`), dizer "põe a URL entre aspas" em vez de falhar torto.
5. **Falhar antes de processar:** toda validação acontece na leitura do Makefile ou nas primeiras linhas do comando — nunca depois de varrer 200 MiB. É a mesma disciplina que já aplicamos em `make dicionario`.

3.10 Exemplo completo, do comando à página

Para `make map QT_MAT_BAS "https://.../estados/35?...&intrarregiao=microrregiao" 2023`:

#	O que acontece	
1	<code>\$(filter http%, \$(MAKECMDGOALS))</code> captura a URL; 2023 vira o ano	R
2	<code>QT_MAT_BAS</code> → natureza contagem → agregação = soma	R
3	<code>intrarregiao=microrregiao</code> → agrupar por <code>CO_MICRORREGIAO</code> (existe em 2023)	R
4	<code>/estados/35</code> → filtrar <code>CO_UF = 35</code>	R
5	<code>carregar_anos([2023], colunas[NU_ANOCENSO, CO_UF, CO_MICRORREGIAO, QT_MATBAS])</code>	—
6	<code>groupby(CO_MICRORREGIAO).sum()</code> → 63 linhas	R
7	malha baixada (cache) e unida por <code>codarea</code> ; cobertura relatada	R
8	grava <code>site/dados/qt_mat_bas-microrregiao-35-2023.geojson</code> + <code>.html</code>	R
9	acrescenta a linha em <code>site/consultas.txt</code> e regenera o <code>index.html</code>	R

4 Arquitetura e arquivos

`microdados (data/interim, data/processed)` [já existe]

```
make map <VAR> "<URL>" [anos] [Python: censo map]
malha do IBGE → data/cache/malhas/<sha1>.json (.gitignore)
agregação → junção por codarea
```

<code>site/dados/<slug>.geojson</code>	← geometria + dado + meta, num arquivo só	(git)
<code>site/mapas/<slug>.html</code>	← página gerada do modelo	(git)
<code>site/consultas.txt</code>	← a lista que define o site	(git)
<code>site/index.html</code>	← gerado da lista	(git)

GitHub Actions publica `site/`

<https://wagnermarques.github.io/censo-escolar--analysis/>

4.1 O que entra no git e o que não entra

Caminho	Git	Por quê
data/cache/malhas/	não	download bruto, grande, reproduzível
data/raw, interim	não	microdados, já ignorados hoje
data/processed/*.csv	não	tabela agregada, inspecionável, reproduzível
site/dados/*.geojson	sim	é o que o Pages serve; pequeno e recortado
site/mapas/*.html	sim	gerado, mas é o produto publicado
site/consultas.txt	sim	o código-fonte do site

4.2 JavaScript

Só o essencial, vendorado em `site/vendor/` (sem CDN, sem build):

Biblioteca	Papel	Por quê
Leaflet	tiles, GeoJSON, popup, zoom	~42 KiB gz; API estável há uma década
<code>fetch()</code> nativo	carregar o <code>.geojson</code>	zero dependência
JS vanilla	escala de cor, legenda, seletor de ano	~100 linhas, num modelo único p/ todos os

Ficam de fora, e o motivo: *PapaParse* (o Python já entrega o arquivo pronto), *D3* (o Leaflet resolve coroplético com um décimo do que há para aprender) e qualquer *framework com build* (a exigência é `git push` e pronto). Única dependência externa em runtime: os tiles do OpenStreetMap.

5 Comandos make

```
make map <VAR> "<URL>" [anos] [OPCAO=v]    # assa um mapa e registra a consulta
make site                                   # reassa tudo a partir de site/consultas.txt
make servir                                # http.server em site/, para ver localmente
make ibge-malhas                           # pré-aquece o cache com as malhas mais usadas
make ibge-malha-municipal                   # idem, só a malha municipal (a mais pesada)
```

`make ibge-malhas` é conveniência, não obrigação: o `make map` baixa o que faltar. Ele existe para quem vai trabalhar sem rede depois, e as malhas podem ser consultadas e baixadas à mão em <https://www.ibge.gov.br/geociencias/organizacao-do-territorio/malhas-territoriais/15774-malhas.html> (Municípios, UFs, Grandes Regiões, Regiões Geográficas Imediatas e Intermediárias, País). O que ele baixa vai para `data/cache/malhas/`, que está no `.gitignore`.

Cada alvo com sua entrada em `make help <alvo>`, e por baixo os subcomandos `censo map`, `censo site` e `censo servir` — a paridade `make/CLI` que o projeto já pratica (funciona no Windows sem `make`).

6 Fatos verificados

Tudo conferido em 2026-08-18, nos dados em disco e nas fontes oficiais.

6.1 Os microdados não têm latitude/longitude

Não existe coluna `LATITUDE`/`LONGITUDE` em nenhum ano. O que há é `CO_MUNICIPIO`, `UF`, endereço em texto e CEP. Coordenadas por escola existem no Catálogo de Escolas, cujo export é um portal de BI **sem URL estável de download** — não dá para automatizar no `make`, e nem toda escola tem coordenada lá.

Decisão registrada: mapas por área (UF, região, município), sem pontos por escola. É o que os dados permitem hoje.

6.2 Até onde o drill-down chega

Você pediu para descer de estado → região → distrito. O que dá e o que não dá:

Nos dados — colunas por ano:

Coluna	Anos
<code>CO_REGIAO</code> , <code>CO_UF</code> , <code>CO_MUNICIPIO</code>	todos (2019–2025)
<code>CO_MESORREGIAO</code> , <code>CO_MICRORREGIAO</code>	2019–2024 (some em 2025)
<code>CO_REGIAO_GEOG_INTERM</code> / <code>_IMED</code>	2023–2025
<code>CO_DISTRITO</code>	todos (2019–2025)
<code>CO_SUBDISTRITO</code>	só 2025

`CO_DISTRITO` é o código IBGE de 9 dígitos (município + 2): 355030840 em São Paulo. **O dado desce até o distrito em todos os anos.**

Na geometria: a API de malhas **não serve distrito** (testado). Existe outra fonte — `geoftp.ibge.gov.br/.../malhas_de_setores_censitarios__divisoes_intramunicipais` —, em shapefile, o que exigiria uma dependência nova (geopandas/fiona) e um alvo `make` à parte.

Os nomes dos distritos, esses são fáceis: `api/v1/localidades/estados/35/distritos` devolve os 1.042 distritos de SP com id de 9 dígitos e nome — junta direto com `CO_DISTRITO`, sem dependência nova.

Portanto: distrito entra primeiro como **painel de dados** (ranking, tabela, barras dentro de um município), não como polígono. O polígono fica para uma fase posterior, se valer a pena.

Sobre "zona leste", "zona norte", "centro": não são divisões do IBGE — não existem nos microdados nem nas malhas. As oficiais em São Paulo são as **subprefeituras**, agrupamentos de distritos definidos pela prefeitura. Podemos ter isso como um arquivo de agrupamento nosso (`site/dados/agrupamentos/sp-zonas.json`: distrito → zona), assumido como convenção do projeto, não como dado oficial.

6.3 O buraco de 2025

A `Tabela_Escola_2025_V2.csv` não tem nenhuma coluna `QT_MAT_*`: em 2025 o INEP moveu as matrículas para a `Tabela_Matricula`, um arquivo à parte. Então: contagem de escolas cobre 2019–2025; matrículas, 2019–2024. O gerador recusa `QT_MAT_BAS` em 2025 com essa explicação (R5).

6.4 Custo das malhas (medido, gzip que o Pages já serve)

Malha	Bruto	Gzip
BR por UF	98 KiB	28 KiB
BR por região intermediária	613 KiB	163 KiB
BR por região imediata	1,1 MiB	283 KiB
BR por município (5.570)	3,6 MiB	817 KiB
SP por município	310 KiB	67 KiB
SP por microrregião (minima)	91 KiB	21,7 KiB
SP por microrregião (maxima)	1,77 MiB	416 KiB

A leitura prática: **recortar por UF é o que barateia tudo** (67 KiB contra 817 KiB), e `qualidade=minima` é o padrão sensato.

6.5 O que o projeto já tem e será reaproveitado

Já existe	Serve para
<code>loading.carregar_anos()</code>	empilhar anos com <code>NU_ANO_CENSO</code> (R5)
<code>loading.carregar_escolas()</code>	ler só as colunas necessárias (Parquet/CSV)
<code>analysis.contar_escolas()</code>	a agregação por área (R4)
esquema / dicionário de dados	natureza, descrição, anos ausentes (R4/R5)
<code>codigos.rotular()</code>	rótulos de <code>TP_*</code> nos popups
<code>amostra.py</code>	conferir o CSV intermediário sem abrir tudo
modificadores do Makefile	anos soltos e validação antecipada (R1/R5/R9)
padrão <code>make help <alvo></code>	ajuda detalhada do <code>make map</code> de graça

7 Decisões registradas

1. **Deploy:** `site/` na main + GitHub Actions (workflow de ~15 linhas com `upload-pages-artifact` + `deploy-pages`). *Escolhido pelo usuário.*
2. **Sem pontos por escola** no v1 — mapas por área, pelos dados que temos.
3. **Malhas baixadas ficam no .gitignore**; o que é publicado é o GeoJSON recortado com o dado dentro (R7).
4. **O construtor de consultas é o make**, não uma tela. O visitante escolhe um mapa pronto na lista.
5. **A URL do IBGE é a linguagem do recorte** — o projeto não inventa vocabulário próprio de nível geográfico.

8 Fases

- **F1** — `make map QT_MAT_BAS ".../paises/BR?...&intrarregiao=UF"` 2023 assa a primeira página, o `index.html` e o workflow do Pages. *Aqui o objetivo já está no ar.*
- **F2** — `site/consultas.txt` + `make site` reassando tudo; mais mapas.
- **F3** — dentro da página: seletor de ano (quando a consulta tem vários) e filtro por dependência, com o dado que o GeoJSON já traz.
- **F4** — distrito como painel de dados (nomes via API de localidades).
- **F5** — se valer: polígono de distrito (shapefile do geoftp) e/ou pontos por escola (Catálogo, export manual).

9 Riscos e limites

- **Dependência da API do IBGE** no momento da geração — mitigada pelo cache (R9.2), que torna `make site` reproduzível offline.
- **Tiles do OSM** são a única dependência externa em runtime; trocar de provedor é uma linha.
- **Malha pesada** se alguém pedir Brasil inteiro por município em `qualidade=maxima` — o aviso da R9.3 existe para isso.
- **Junção incompleta** entre malha e dados (municípios criados/extintos entre edições): por isso a cobertura é relatada, não presumida (R3).
- **2025 sem matrículas** na tabela de escolas.

10 Critérios de verificação

- `make map QT_MAT_BAS "...&intrarregiao=UF" 2023` gera os dois arquivos e o total do GeoJSON bate com `analysis.panorama_por_uf()` para o mesmo ano;
- a cobertura relatada é 27/27 áreas (UF) e 63/63 (microrregiões de SP);
- `make servir` + navegador: o índice lista o mapa, o mapa desenha, o popup mostra nome e valor;
- `make site` a partir de um `consultas.txt` com três linhas reproduz três páginas idênticas às geradas uma a uma;
- depois do push, a URL do Pages mostra o mesmo resultado.

11 Histórico das decisões

Três rodadas de conversa, e o que cada uma descartou:

1. **Páginas fixas** (proposta inicial): um HTML por tema, escrito à mão. Descartado por não escalar para as ~467 variáveis mapeáveis.
2. **Construtor de consultas na tela** (ideia do usuário, 2026-08-18): a URL da página como consulta, com formulário gerado de um `manifesto.json`. Descartado depois do esclarecimento seguinte — mas deixou duas ideias que sobreviveram: a consulta como texto e o índice como lista de consultas salvas.

3. **O construtor é o make** (esclarecimento do usuário): quem monta a consulta é quem gera o site. Some o manifesto, some a UI, some o risco de URL sem arquivo, some a explosão combinatória.
4. **A URL da malha é a consulta espacial** (proposta do usuário): a linguagem do IBGE substitui a nossa. Some o vocabulário próprio de nível, some o alvo **make malha** separado, e o comando fica com dois argumentos.

O que sobreviveu de ponta a ponta: *o site é função de uma lista de consultas, e o Python calcula, o navegador só desenha.*